

Implementation and Study of Evolutionary Algorithms Compared to Non-evolutionary Methods

April 17, 2026



Student: Jair Mendoza

Id: 269615

Instructors: Marcin Komarnicki, Konrad Gmyrek

Class: Optimization Methods: Theory and Applications

Content

Introduction.....	2
PFSP.....	3
Random search.....	3
Greedy algorithm.....	3
Simulated annealing.....	4
Genetic algorithm (GA).....	4
Implementation.....	6
PFSP.....	6
Random search.....	7
Greedy algorithm.....	7
Simulated annealing.....	8
Genetic algorithm.....	8
Results.....	11
Mutation probability.....	11
Crossover probability.....	12
Population size.....	12
Tournament size.....	13
Conclusions.....	13
Comparisons.....	13
Tai20_5_0.....	14
Tai20_10_0.....	15
Tai20_20_0.....	15
Tai100_10_0.....	16
Tai100_20_0.....	16
Tai500_20_0.....	17
Convergence graph.....	17
SA parameters.....	18

Introduction

The following descriptions of the topics are done in a simple and general way since the objective of this report is to show the implementation of them.

PFSP

The Permutation Flowshop Scheduling Problem (PFSP) is an NP-hard problem, the core idea of the problem is that we have J jobs that should be performed in M machines. Each job has a specific computational time for each machine. The goal is to find a job sequence that gives us the lowest possible time needed for the jobs to be performed in all machines. The main constraint is that for a j job to be processed on a m machine, the previous j job must be completed by the previous m machine first.

This report handle cases where the values are always for $j > 1$ and $m > 1$ thus the formula for the completion time is as follows:

$$c(xj, m) = \max\{c(xj - 1, m), c(xj, m - 1)\} + p(xj, m) \quad (1)$$

where $c(xj, m)$ is the completion time of the xj th job on the m th machine and $p(xj, m)$ denotes the processing time of job xj on machine m .

This report used the dataset provided by Taillard, specifically:

- tai20_5_0, tai20_10_0, and tai20_20_0.
- tai100_10_0 and tai100_20_0.
- tai500_20_0.

Random search

As its name suggests, this algorithm iterates over randomly selected solutions from the solution space (all possible sequences of jobs) and keeps the best one. This is the simplest algorithm used in the report.

Greedy algorithm

Greedy algorithm is basically a random search algorithm but implemented on each node of the solution rather than to the whole of it. Let a solution be a sequence of nodes $S = (n_1, n_2, n_3, \dots, n_k)$, if the algorithm starts at the node n_1 the next node would be described as:

$$n_{i+1} = \min(w(n_i, v))$$

where $w(n_i, v)$ is the weight and v is the set of remaining nodes. Due to its nature, this algorithm is deterministic.

Simulated annealing

This algorithm works very similar to the hill climb algorithm, it is basically the same but with an additional parameter for accepting worse solutions. It starts by choosing a random solution from the solution space, then it proceeds to check neighbours solutions, if one is better it immediately becomes the next solution but if it is worse then it may be accepted if the probability allows it. The probability is given by formula:

$$p = e^{-\Delta E/T} \quad (2)$$

where ΔE is the difference between solution values and T is the heat parameter. Each iteration the probability T decreases based on the parameter which is usually called temperature.

If we visually imagine the algorithm giving jumps between the solution space, then the higher the T is the bigger the jumps it will take as the next step and as T is decreasing (cooling down), the jumps become smaller. This behaviour is great to avoid local minimums which is the exact problem the Hill climbing algorithm has.

Neighbour: the term refers to a small change in the solution. If the solution is a permutation in the form of $x = \{a, b, c, d, e\}$ then a neighbor is simply a swap in the order $x' = \{a, d, c, b, e\}$.

Genetic algorithm (GA)

This is the main focus of this report. The genetic algorithm GA works by choosing and adjusting few parameters:

Population: Selected solutions that will act as parents.

Generations: Number of generations we want to have.

Selection parameters: The main ways for parent selection are tournament and roulette.

Mutation probability: How frequent the mutation is going to happen.

A genetic algorithm is defining individuals(solutions), combining them and generating new promising solutions. The algorithm mimics in a general way how natural selection or evolution works.

The usual flow of the algorithm is: First, the population is initialized. Parent candidates are then chosen; for example, in Tournament Selection, a fixed number of solutions are randomly picked from the population, and the best among them is selected. This is repeated until the desired number of parents is reached.

Secondly we generate the childs. Two parents are chosen based on the crossover operator. There are two types of crossover: OX (Ordered Crossover) which is the easiest to implement and PMX (Partially Matched Crossover) which is a bit more complex. OX works by taking a random sequence from Parent 1 and copies it to the same position in the child. The remaining spots are

filled using the sequence from Parent 2. This process respects specific problem constraints, such as preventing duplicate values in the sequence.

Lastly we have mutation, this can be of two forms Swap and Inversion. For swap, two genes in a chromosome are selected and its values are swapped. In inversion, two random positions in the sequence are selected, and the order of the chromosomes within this selected segment is reversed.

Implementation

The implementation was initially done in Python and later was moved to C++. Python was used to quickly sketch the algorithms and C++ to save time on big datasets (like tai500_20_0.fsp).

PFSP

The presented function code implements the formula (1):

```
Python
subject = [Job(id=i, times=t) for i, t in enumerate(subject)]
def solution(jobs, machines):
    time = [0] * machines
    for jb in jobs:
        time[0] += jb.times[0]
        for i in range(machines):
            time[i] = max(time[i], time[i - 1]) + jb.times[i]
    return time
sol = solution(subject, machines)
```

As it is visible the subject is an indexed list of jobs ids and its processing times. This was a bit confusing to work with, that is why in the c++ version it is handled quite differently. In c++ we are working with a class named Subject_solution that have a method to calculate the makespan:

```
C/C++
void compute_makespan() {
    int num_machines = (int)this->permutation[0].times.size();
    std::vector<int> time(num_machines, 0);
    for (const auto& job : this->permutation) {
        time[0] += job.times[0];
        for (int i = 1; i < num_machines; i++) {
            time[i] = std::max(time[i], time[i - 1]) + job.times[i];
        }
    }
    this->makespan = time[num_machines - 1];
}
```

In the c++ version, the class Subject_solution handles the indexing, jobs, times, machines, making the algorithm's syntax readability much more efficient (the python version also implements a class but it was much poorly handled). The python version returns a list of makespan of each job while the C++ computes only the total makespan .

Random search

This implementation is pretty much straightforward, it starts the loop and shuffles the given sequence and computes the makespan. This makespan is compared with the Highest_solution found and updates it if it finds a new one. It will return the best sequence of jobs and makespan.

```
Python
def randomSearch(subject, machines):
    Highest_solution = float("inf")
    Best_subject = None
    for i in range(10000):
        rand.shuffle(subject)
        sol = solution(subject, machines)
        if Highest_solution > sol[-1]:
            Highest_solution = sol[-1]
            Best_subject = subject.copy()
    return Best_subject, Highest_solution
```

Greedy algorithm

Here we remove the first element of the job sequences and pass the removed element to another list named *Best_subject*. The algorithm then starts to compare which elements of the remaining solutions together with the *Best_subject* gives the best makespan and appends the element which has the best makespan and repeats this until there are no more remaining elements.

```
Python
def greedy(subject, machines):
    temp_subject = subject.copy()
    Best_subject = [temp_subject.pop(0)]
    rand.shuffle(temp_subject)
    while temp_subject:
        current_sol = float("inf")
        current_job = None
        for s in temp_subject:
            current_step = Best_subject + [s]
            ms = solution(current_step, machines)[-1]
            if ms < current_sol:
                current_sol = ms
                current_job = s
        Best_subject.append(current_job)
        temp_subject.remove(current_job)
    return Best_subject, current_sol
```

Simulated annealing

There is a function made for the genetic algorithm for swapping a given solution which is being used here to reproduce neighbours. The code sets an initial solution and iterates over its neighbours. The condition for accepting a worst solution is by generating a random number between 0-1 and comparing it with the probability given by the formula (2) (if the probability is 0.9 then there is 90% chance for the generated number to be in this range).

```
Python
def simulatedAnnealing(
    subject, machines, temp=500, cooling_rate=0.999, iterations=10000):
    rand.shuffle(subject)
    current_sol = solution(subject, machines)[-1]
    current_subject = Subject_solution(subject, current_sol)
    for _ in range(iterations):
        temp_subject = current_subject.copy()
        swap(temp_subject, machines)
        if temp_subject.makespan < current_sol:
            current_sol = temp_subject.makespan
            current_subject = temp_subject.copy()
        elif rand.random() < 2.71828 ** ((current_sol-temp_subject.makespan)/temp):
            current_sol = temp_subject.makespan
            current_subject = temp_subject.copy()
        temp *= cooling_rate
    return current_subject
```

After each iteration the temperature value is decreased and updated. For better handling of the data in this and the next algorithm the class `Subject_solution` that manages the solution, makespan and ids of jobs is used.

Genetic algorithm

Individuals are the representation of the problem solution. Parents and children are individuals. For this implementation the individual is an instance of a class defined in the code. Since the experiment was done in two languages there are two different classes but the idea is the same.

Individual classes have *job time permutation* and *makespan order* properties (C++ have indexes of the permutation and have total makespan instead of makespan order). In general all the methods are applied to the job index permutations to have more clarity.

The method used to generate childs is the *ox ordered crossover*, the parents P1&P2 are objects of the `Subject_solution` class. Ox takes the indexes of the job sequence in the solution of parent P1 and proceeds to randomly take a portion from it and paste it into the child, then it iterates over the indexes of the parent P2 and copies them into the child unless the job is already there.

The mutation is of swap type, the logic is simple we just take two jobs from the solution, swap their values and calculate the new makespan. These two functions(ox and mutation) are methods in the C++ class version of the code. This in order to maintain more clarity among the code.

Below are the functions and class code in python:

Python

```
class Subject_solution:
    job_lookup = {}
    def __init__(self, permutation, makespan):
        self.permutation = permutation
        self.makespan = makespan
    def copy(self):
        return Subject_solution(self.permutation.copy(), self.makespan)
    @classmethod
    def set_lookup(cls, subject):
        cls.job_lookup = {job.id: job for job in subject}
    @classmethod
    def from_ids(cls, child_ids, machines):
        perm = [cls.job_lookup[id] for id in child_ids]
        ms = solution(perm, machines)[-1]
        return cls(perm, ms)
    @property
    def ids(self):
        return [job.id for job in self.permutation]
```

from_ids() function is used to generate an object based on the index sequence.

Python

```
def ox(parent1, parent2, machines):
    child_ids = [None for _ in range(len(parent1))]
    i = rand.randint(0, len(parent1) - 1), j = rand.randint(0, len(parent1) - 1)
    if i > j:
        i, j = j, i
    child_ids[i:j] = parent1[i:j]
    for gene in parent2:
        if gene not in child_ids:
            for k in range(len(child_ids)):
                if child_ids[k] is None:
                    child_ids[k] = gene
                    break
    return Subject_solution.from_ids(child_ids, machines)
def swap(subject, machines):
    i=rand.randint(0, len(subject.ids) - 1), j=rand.randint(0, len(subject.ids) - 1)
    if i > j:
        i, j = j, i
    subject.permutation[i], subject.permutation[j] = (
        subject.permutation[j],
        subject.permutation[i],)
    subject.makespan = solution(subject.permutation, machines)[-1]
```

Once these functions are implemented the GA flow becomes much more easy to understand. First the code initializes the population which is just a list of Subject_solution objects, then the tournament selection is started which overwrites the parents into the population list. Secondly, we take the parents and start producing the child by taking parents in pairs. Finally, we iterate through all the children and mutate them according to the mutation probability.

Python

```
def genetic(subject,machines,population_size=100,generations=100,
            root_parents=20,tournament_size=10,pm=0.1,):
    Population = []
    population_makespan = []
    temp_subject = subject.copy()
    for i in range(population_size):
        ... # generates population
    for gen in range(generations):
        Parents = []
        Children = []
        # Tournament selection
        for _ in range(root_parents):
            tournament = rand.sample(Population, tournament_size)
            best_makespan = float("inf")
            best_parent = None
            for individual in tournament:
                if individual.makespan < best_makespan:
                    best_makespan = individual.makespan
                    best_parent = individual
            Parents.append(best_parent)
        # Crossover ox
        for i in range(0, len(Parents), 2):
            if i + 1 < len(Parents):
                Children.append(ox(Parents[i].ids, Parents[i + 1].ids, machines))
                Children.append(ox(Parents[i + 1].ids, Parents[i].ids, machines))
        # Mutation swap
        for child in Children:
            if rand.random() < pm:
                swap(child, machines)
        Population = Children.copy()
    for s in Population:
        ... # picks the best subject from the population
    return Population, best
```

Results

The datasets and parameters used in the report were:

	Genetic algorithm				Simulated Annealing	
	Population size	Generations	Mutation prob.	Swap prob.	Temperature	Cooling rate
Tai20_(5,10,20)_0.fsp	180	100	0.85	1	150	0.9
Tai100_(10,20)_0.fsp	180	100	0.85	1	150	0.9
Tai500_20_0.fsp	360	50	0.85	1	150	0.9

The code ran and tested 20 instances of 18k iterations for each algorithm; the time difference between python and c++ was not very noticeable in small data but was clearly different in the hard dataset. Below is a comparison between the completion times it takes on average.

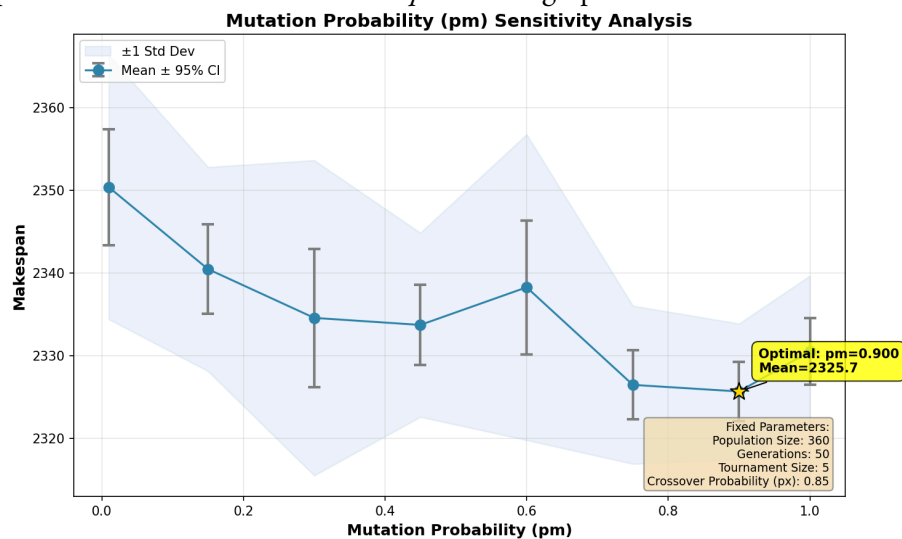
	Tai20_20_0.fsp [m]	Tai100_20_0.fsp [m]	Tai500_20_0.fsp [m]
C++	~0m28s	~3m	~20m
Python	~0m54s	~4m50s	~55m23s

For the GA algorithms the parameters that can be tuned and its default values are the following: These parameters have been tuned and their influence has been collected with 20 instances of the genetic algorithm over the Tai20_20_0.fsp and Tai500_20_0 dataset.

In order to have a fair comparison between the population size, the iteration budget has been set to 18k. For the Tai500_20_0 the default population size = 360 and generation = 50.

Mutation probability

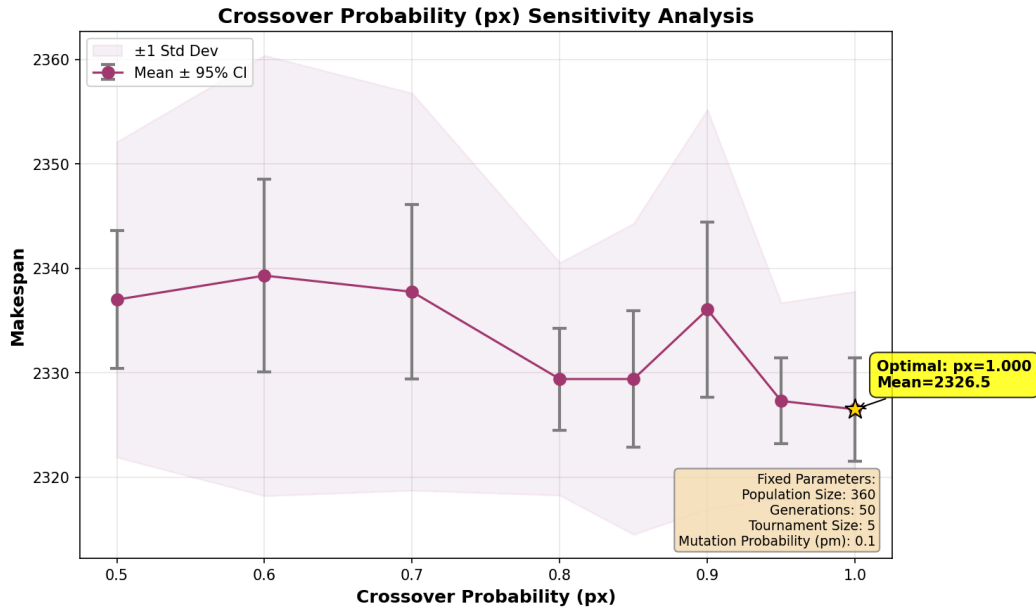
Below is a graph that shows the influence of the pm on the graph.



The graph shows that very high mutation probabilities are acceptable and indeed they give a better result. In this GA implementation we are changing only two places of our individual per mutation, this means that even 100% would be acceptable since it is not too destructive.

Crossover probability

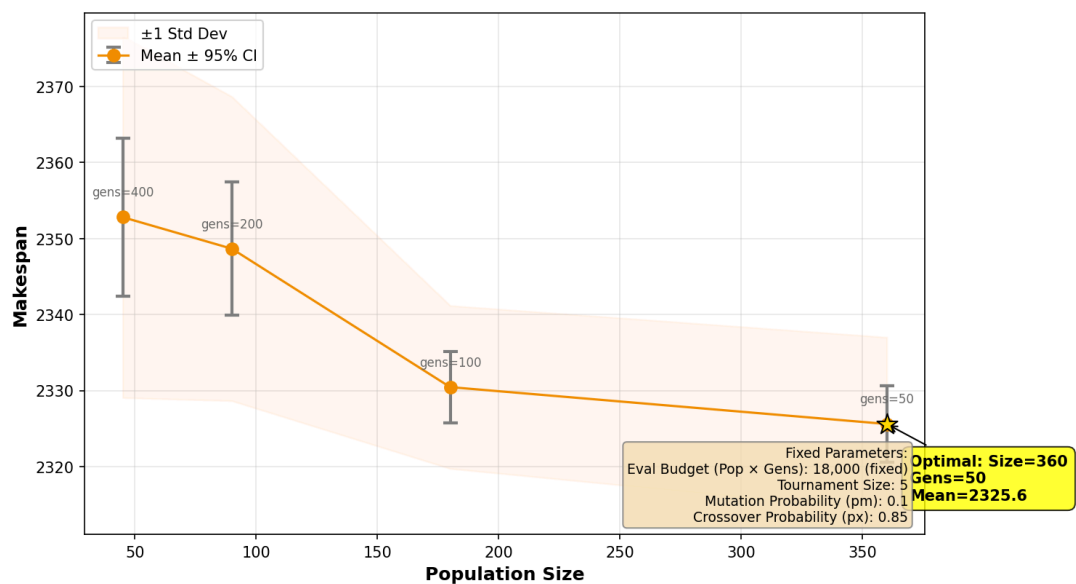
Taking $pm=0.85$, $tournament\ size=2$, the influence of crossover probability px look like follows:



From the chart it can be said that the best range is between $0.8 \leq px \leq 1$. It is understandable due to the fact that the code also implements an elitist approach after the kids have been generated (there is a 20% that the parents will still be present in the next generation together with the children). So in the current approach there is still a chance for the parents to be present even after the children's generation through px .

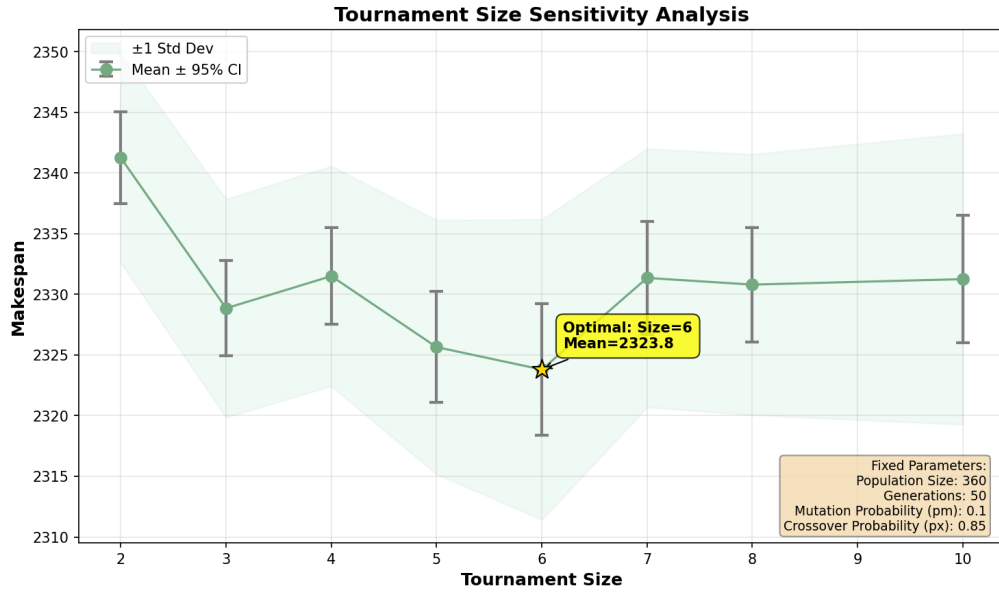
Population size

The following charts take $pm=0.85$, $px=1$ and $tournament\ size=5$. Best range is: $180 \leq ps \leq 400$



Tournament size

The following charts takes $pm=0.5$, $px=1$ and, $population\ size=360$:



Here the best value is $6 \leq tournament \leq 10$, being $tournament=6$ the most reliable

Conclusions

The GA in general performs worse than the SA. Below are performance comparisons between all of the algorithms. Regarding GA, in some cases (such as the tai500_20_0 dataset), the chosen parameters may differ from the values proposed by the analysis of each parameter's influence on that particular dataset. This is mainly to save computational time, as the improvements achieved with the optimal parameters are not worth the additional cost, as shown in the next sections.

For the comparisons, two instances of 10 runs were run for each dataset.

Comparisons

The parameters used for the Genetic Algorithm are:

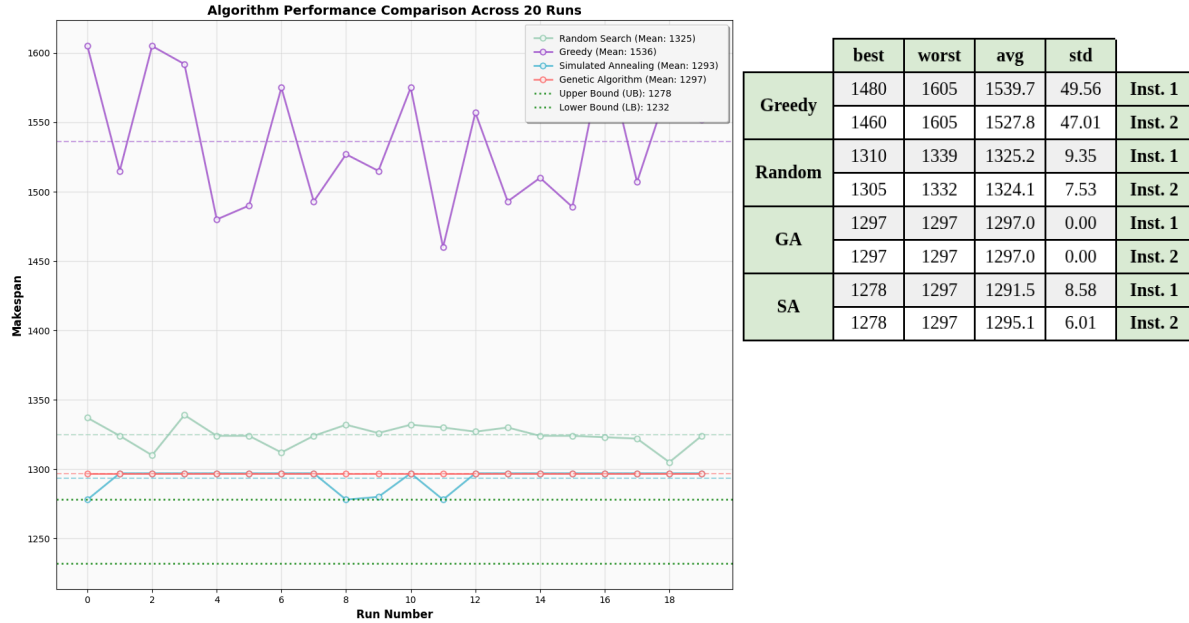
- pop size (*population size*) = 180.
- gen (*number of generations*) = 100.
- Px (*crossover probability*) = 1.
- Pm (*mutation probability*) = 0.85.
- Tour (*tournament size*) = 2.

For the Simulated Annealing are:

- T (*Temp*) = 100.
- *cooling_rate* = 0.95.

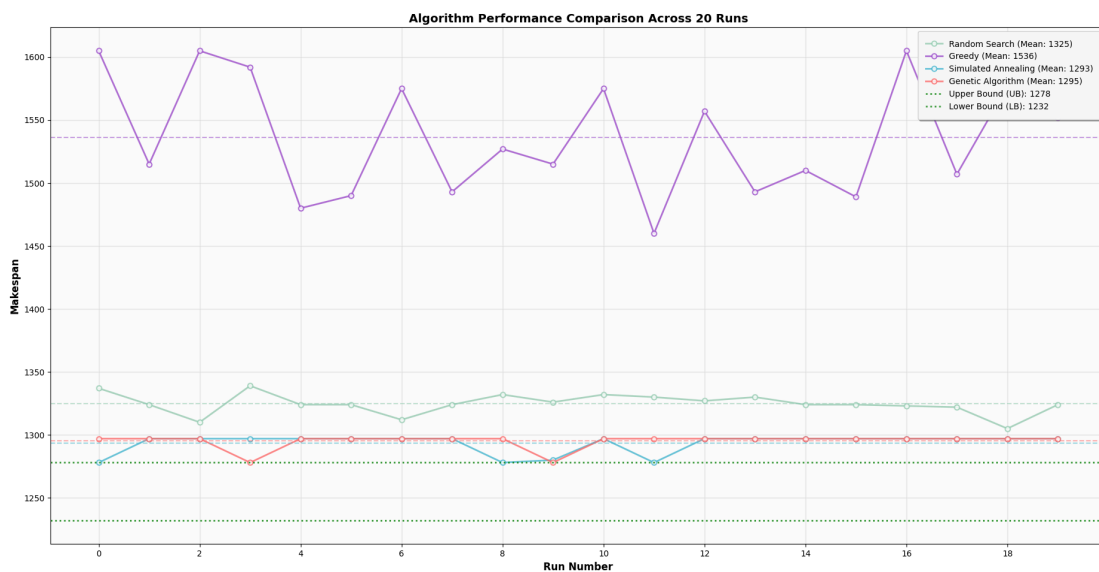
Tai20_5_0

This comparison is for 20 jobs and 5 machines. The greedy algorithm will start from a random initial condition but it still remains deterministic.



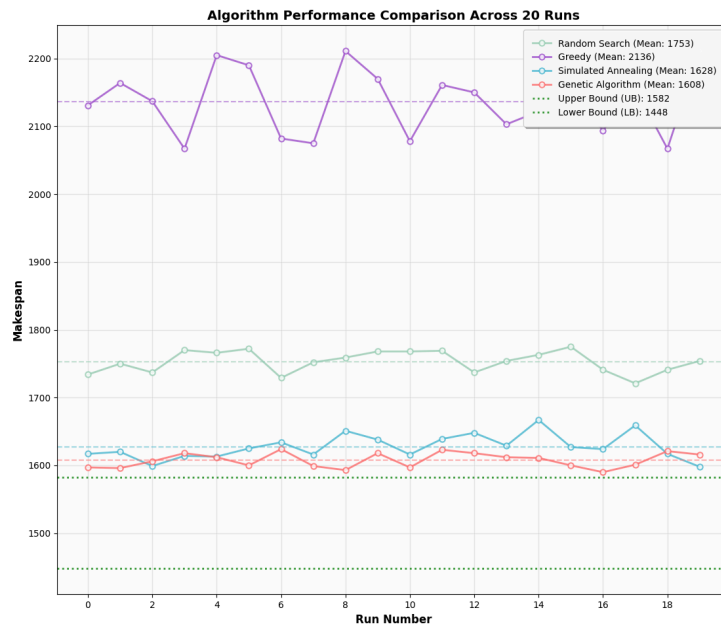
Here it is visible how the two metaheuristic algorithms consistently find a very good solution. It is worth noting that in this run only SA was able to escape the local minima. Another clear point for this dataset is that the genetic algorithm is the most reliable and consistent. On the other hand, the SA algorithm whenever goes outside the average is to always give better results.

It is interesting to note that just by changing the parameters a little the solution can improve, in this case changing both $pm=0.5$ and $px=1$ to 0.6 and $tournament=5$ to 2 . It can be said that the old configuration resembled more to exploration but given that this dataset is considerably simple, the best approach would be to have a more exploitation-focused approach.



Tai20_10_0

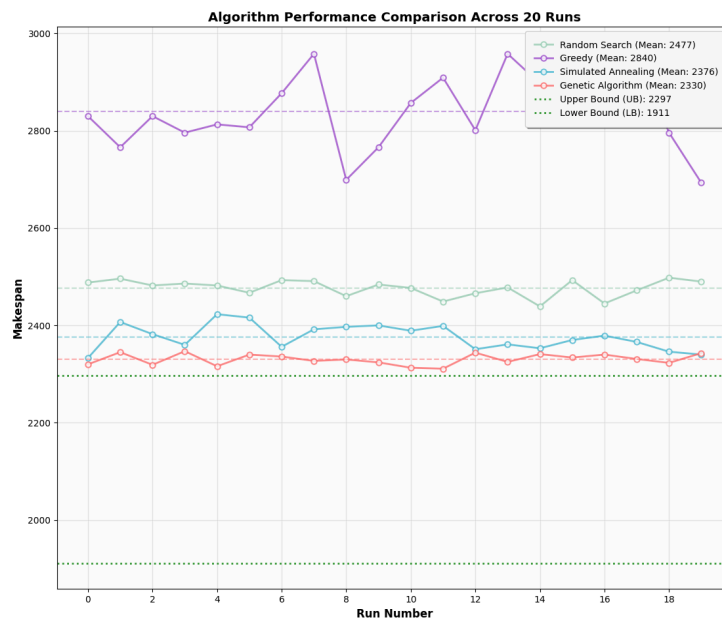
This comparison is for 20 jobs and 10 machines, so the possible combinations remain the same (that is 20!) yet the complexity for finding an optimal makespan is more complex. Here, the GA is by far the best algorithm. It has the best solution, average and deviation.



	best	worst	avg	std	
Greedy	2067	2211	2143.2	53.90	Inst. 1
	2067	2211	2135.4	44.70	Inst. 2
Random	1729	1772	1753.7	15.91	Inst. 1
	1721	1775	1752.3	17.09	Inst. 2
GA	1593	1624	1606.3	10.97	Inst. 1
	1590	1623	1608.9	11.22	Inst. 2
SA	1599	1651	1622.7	14.23	Inst. 1
	1598	1667	1632.4	21.06	Inst. 2

Tai20_20_0

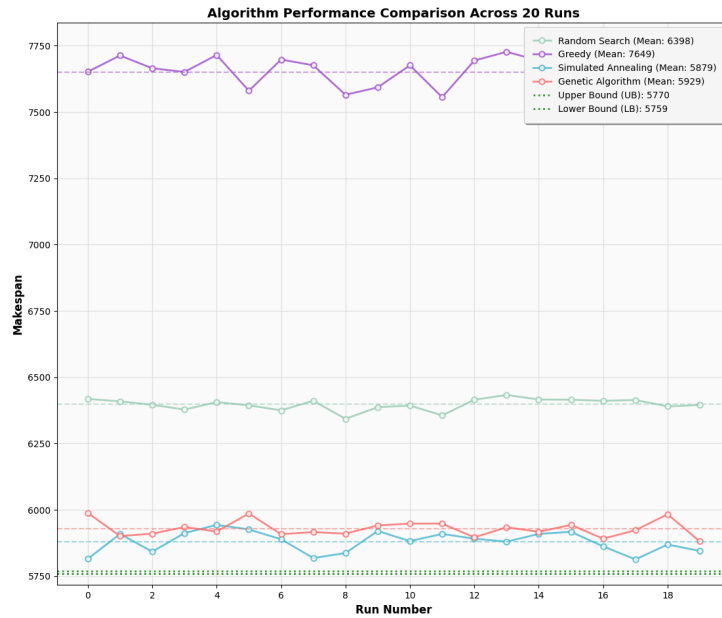
This comparison is for 20 jobs and 20 machines. Again the GA is by far the best algorithm. The default configuration helps GA to escape minimum and get to a better solution.



	best	worst	avg	std	
Greedy	2699	2958	2814.2	69.46	Inst. 1
	2693	2961	2866.7	87.34	Inst. 2
Random	2460	2496	2482.9	11.33	Inst. 1
	2439	2498	2470.7	20.73	Inst. 2
GA	2316	2347	2330.4	11.11	Inst. 1
	2311	2344	2330.5	12.13	Inst. 2
SA	2333	2423	2386.6	27.32	Inst. 1
	2340	2399	2365.4	19.13	Inst. 2

Tai100_10_0

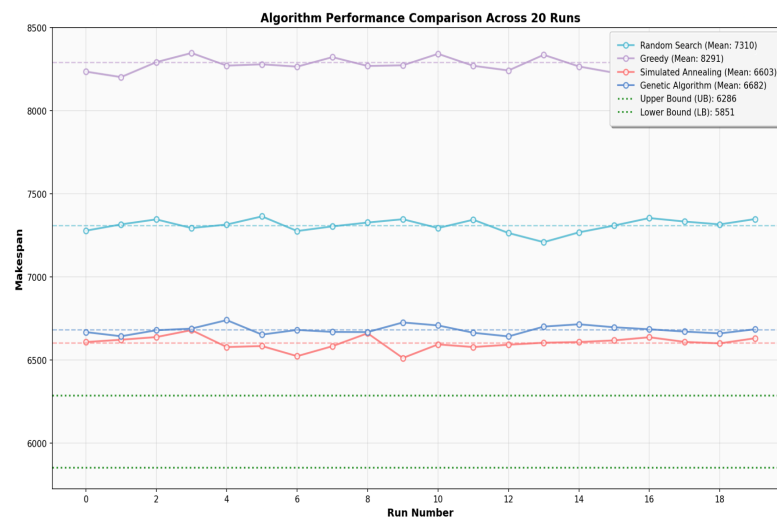
This comparison is for 100 jobs and 10 machines. The simulated annealing performs better than the genetic algorithm although the difference is very small. Still GA keeps more consistent.



	best	worst	avg	std	
Greedy	7565	7715	7650.8	54.54	Inst 1
	7514	7740	7642.7	77.87	Inst 2
Random	6343	6418	6391.7	22.24	Inst 1
	6356	6433	6403.8	21.29	Inst 2
GA	6609	6754	6694.9	54.90	Inst 1
	6674	6731	6696.5	18.26	Inst 2
SA	5815	5943	5881.1	48.28	Inst 1
	5813	5917	5877.7	32.11	Inst 2

Tai100_20_0

This comparison is for 100 jobs and 20 machines. The SA is better than GA both in values and reliability.

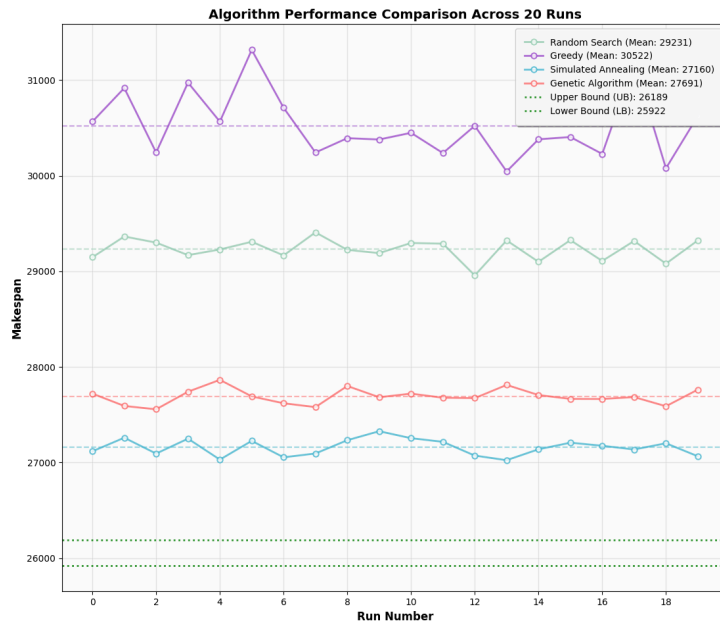


	best	worst	avg	std	
Greedy	8202	8347	8275.5	40.53	Inst 1
	8228	8374	8304.3	50.88	Inst 2
Random	7244	7336	7288.3	39.15	Inst 1
	7209	7325	7285.1	34.23	Inst 2
GA	6609	6754	6694.9	54.90	Inst 1
	6674	6731	6696.5	18.26	Inst 2
SA	6502	6627	6557.1	40.62	Inst 1
	6536	6592	6574.7	17.84	Inst 2

Here the parameters were different: population size = 250, tour size = 7, pm = 0.5 and px = 0.95.

Tai500_20_0

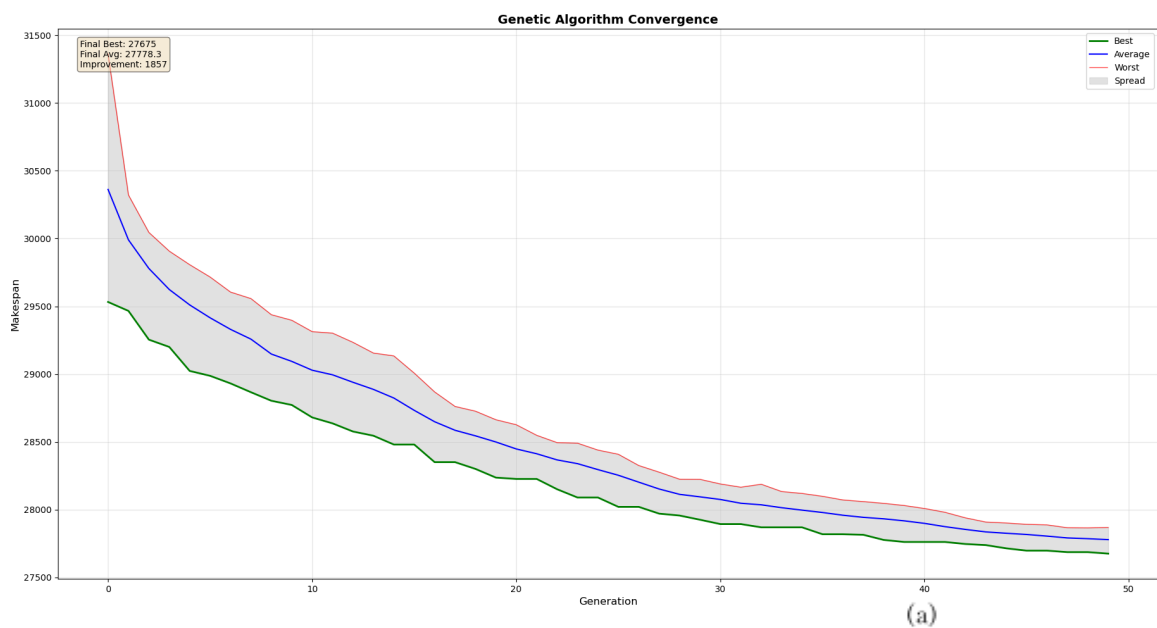
This comparison is for 500 jobs and 20 machines. For the first time, the Random search algorithm is closer in performance to the greedy algorithm than to both metaheuristic algorithms. The main cause may be that random search in such a big solution space(500!) has more probabilities to start at a bad point and get easily stuck.



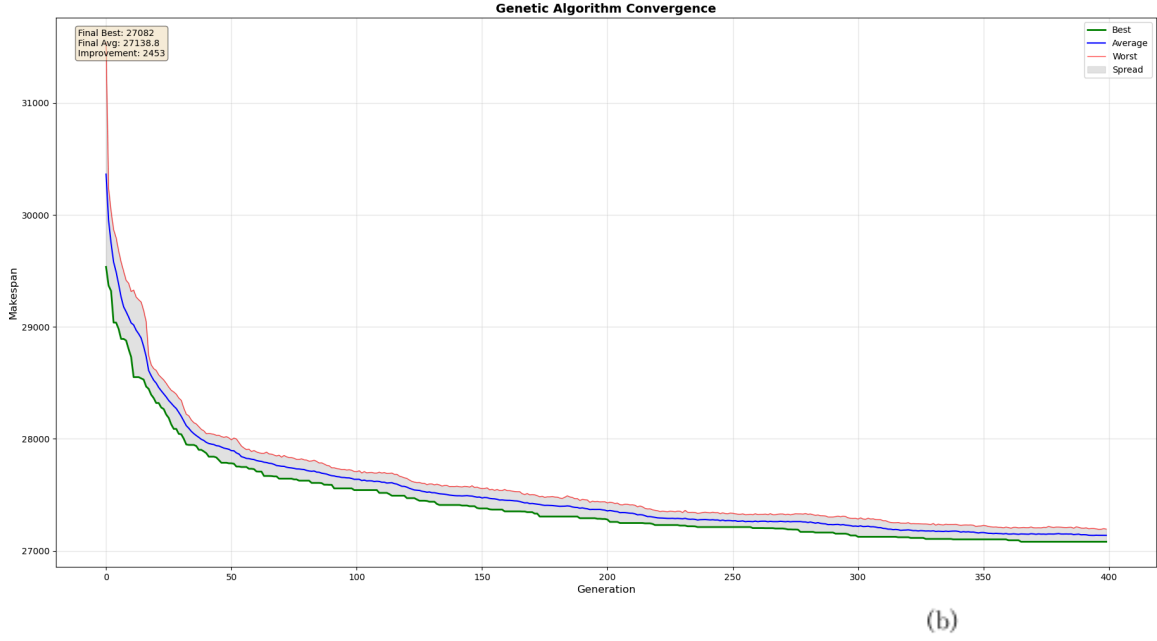
	best	worst	avg	std	
Greedy	30242	31315	30629.8	350.36	Inst. 1
	30044	31179	30410.1	326.58	Inst. 2
Random	29147	29407	29250.4	89.82	Inst. 1
	28957	29326	29211.8	136.50	Inst. 2
GA	27557	27865	27685.2	100.35	Inst. 1
	27589	27812	27695.9	60.47	Inst. 2
SA	27031	27328	27169.2	100.07	Inst. 1
	27025	27255	27149.8	75.39	Inst. 2

Convergence graph

Below are two GA convergence graphs: (a) and (b), both for different configurations.



Configuration for (a): population=360, generations=50, tournament=6, $P_m=1$, $P_x=0.9$.

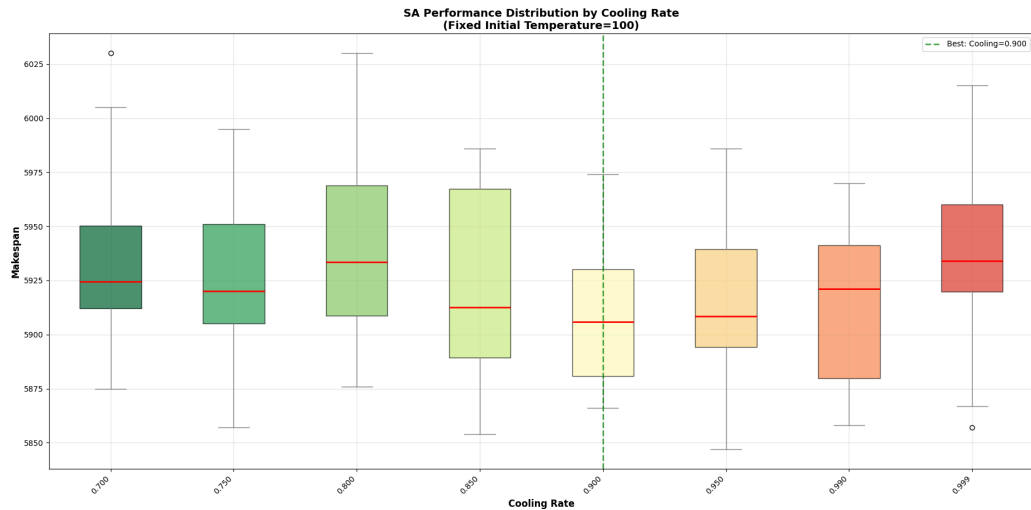


Configuration for (b): population=360, generations=400, tournament=6, $P_m=0.9$, $P_x=1$.

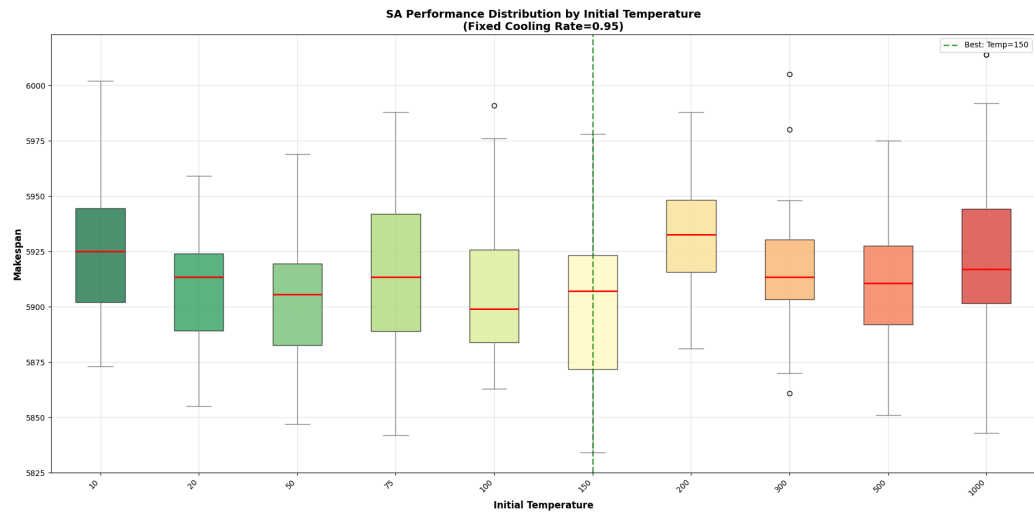
In general terms, between the two (a) and (b) configurations. (a) converges faster than (b). Regarding configuration (b), its generations improve significantly between 300-600 whereas the improvement for (a) in this range is almost insignificant. However, taking into account that the average value achieved by the configuration (a)=27778 and for (b)=27139, the improvement achieved by the configuration(b) over (a) is just around $\sim 2.3\%$, so to save time the report used the (a) configuration

SA parameters

This algorithm was the one that generally performed the best across all the datasets. It has two main parameters, Temperature and Cooling rate. Below are the graphs that were used to get the best parameters.



This graph shows that the best range for *cooling rate* is between 0,9 - 0,99. The value chosen was 0.9 since it is clear that it is more consistent than the others.



Looking at the graph the value picked for the *temperature* was 150, even though it is not the most consistent yet is the point where it can give the best solutions and the worst solutions possible are not as bad as other parameters.